

In this document, I will list the 12 bugs in the transformer, explain, and fix.

```
1h = self.tok(x) + self.pos(x)
```

x contains token IDs, not positions.

```
1 # Fix 1
2 positions = torch.arange(S, device=x.device).unsqueeze(0)
3 h = self.tok(x) + self.pos(positions)
```

```
2q = q.view(B, S, H, D)
2k = k.view(B, S, H, D)
3v = v.view(B, S, H, D)
```

We want to shape them instead as (B, H, S, D) .

```
1 # Fix 2
2 q = q.view(B, S, H, D).transpose(1, 2)
3 k = k.view(B, S, H, D).transpose(1, 2)
4 v = v.view(B, S, H, D).transpose(1, 2)
```

```
3attn = torch.matmul(q, k.transpose(-2, -1))
```

Each token should attend over previous sequence tokens, not heads. Building on Fix 2,

```
1 # Fix 3
2 attn = torch.matmul(q, k.transpose(-2, -1))
```

now has shape (B, H, S, S) .

```
4attn = torch.matmul(q, k.transpose(-2, -1))
```

Attention should be scaled down by square root of heat dimension, otherwise logits are too large.

```
1 # Fix 4
2 attn = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(D)
```

```
5attn = torch.matmul(q, k.transpose(-2, -1))
```

Causal model should not look into the future. Create a mask to make this causal:

```
1 # Fix 5
2 mask = torch.tril(torch.ones(S, S, device=h.device, dtype=torch.bool))
3 attn = attn.masked_fill(~mask, float("-inf"))
```

```
6w = F.softmax(attn, dim=-2)
```

Softmax should have dimensional over keys

```
1 # Fix 6
2 w = F.softmax(attn, dim=-1)
```

```
7z = torch.matmul(w, v).reshape(B, S, -1)
```

Before reshaping, we need to move the sequence dimension back to the head dimension.

```
1 # Fix 7
2 z = torch.matmul(w, v)
3 z = z.transpose(1, 2).contiguous().view(B, S, self.d_model)
```

```
81 h = self.proj(z)
```

Transformer blocks should use residual connections:

```
1 # Fix 8
2 h = h + self.proj(z)
```

```
91 h = h + self.ff2(self.ff1(x))
```

Transformer MLPs need a nonlinear activation, usually GELU or ReLU.

```
1 # Fix 9
2 h = h + self.ff2(F.gelu(self.ff1(x)))
```

```
101 return self.head(h).softmax(-1)
```

F.cross_entropy internally applies *log_softmax*.

```
1 # Fix 10
2 return self.head(h)
```

```
111 return torch.optim.SGD(model.parameters(), lr=10.0)
```

Training rate is too large, may face explosion.

```
1 # Fix 11
2 return torch.optim.AdamW(model.parameters(), lr=3e-4, weight_decay=0.01)
```

```
121 logits = model(seq)
2 loss = F.cross_entropy(
3     logits.reshape(-1, logits.size(-1)),
4     seq.reshape(-1),
5 )
6 opt.step()
7 loss.backward()
```

This still predicts the current step, not the next one.

```
1 # Fix 12
2 opt.zero_grad()
3 x = seq[:, :-1]
4 y = seq[:, 1:]
5 logits = model(x)
6 loss = F.cross_entropy(
7     logits.reshape(-1, logits.size(-1)),
8     y.reshape(-1),
9 )
10 loss.backward()
11 opt.step()
```